

Урок 8. JS — объекты и массивы

Объект

Объект — способ хранения и структурирования данных в JS. Он объединяет разные типы данных в одну логическую единицу — например массивы, функции. В объекте удобно хранить данные и управлять ими. Объект — это набор свойств (ключей), где каждое свойство имеет имя и значение.

Способы создания объекта:

1. Литерал объекта, через фигурные скобки:

```
const user = {  
  name: "Марат",  
  age: 30,  
  city: "Москва",  
};  
  
console.log(user);
```

2. Через конструктор `Object()`:

```
const person = new Object();  
person.name = "Марат";  
person.age = 30;  
person.city = "Москва";  
  
console.log(person);
```

Доступ к свойствам объекта

Осуществляется двумя способами:

1. Точечная нотация:

```
console.log(user.name);
```

2. Скобочная нотация с динамическим доступом — позволяет получить доступ к свойствам, в том числе с пробелами в имени:

```
const user = {  
  name: "Марат",  
  "last name": "Муслимов",  
  age: 30,  
  city: "Москва",  
};  
  
console.log(user["last name"]);
```

Скобочную нотацию также используют для динамического доступа — то есть в зависимости от выбора пользователя:

```
const field = prompt("Что показать? (name, last name, age, city)");
console.log(user[field]);
```

Object.keys(), Object.values(), Object.entries()

Эти методы позволяют получить содержимое объекта в виде массива — когда нужно перебрать все свойства или вывести их.

- `Object.keys(user)` — возвращает массив имён свойств объекта.
- `Object.values(user)` — возвращает массив значений свойств объекта.
- `Object.entries(user)` — возвращает массив пар [имя, значение] для каждого свойства.

```
console.log(Object.keys(user));
console.log(Object.values(user));
console.log(Object.entries(user));
```

Сравнение объектов

Объект хранит не значение, а ссылку на место в памяти. Поэтому при сравнении двух объектов с одинаковыми значениями результатом будет `false` — у них разные ссылки.

Копирование объекта

1. Через `Object.assign`:

```
const user1 = Object.assign({}, user);
```

2. Через оператор расширения `...` (spread) — достаёт все свойства из объекта:

```
const user2 = { ...user1 };
```

На основе `user1` создаётся новый объект, и в него копируются свойства. Но если внутри есть вложенный объект — он останется общей ссылкой на один и тот же вложенный объект.

3. Полное (глубокое) копирование — через `structuredClone`:

```
const user3 = structuredClone(user1);
```

Деструктуризация объектов

Деструктуризация — способ распаковать свойства объекта в отдельные переменные.

Базовая деструктуризация:

```
const { name, age, city } = user3;
// теперь name, age, city — отдельные переменные
// со значениями соответствующих полей объекта user3
```

С оператором `rest ...` — остальные свойства собираются в отдельный объект:

```
const { name, age, ...rest } = user3;  
// name и age — отдельные переменные,  
// rest — объект со всеми остальными свойствами
```

С переименованием переменных:

```
const { name: userName, age: userAge } = user3;  
// значения свойств name и age сохранены  
// в переменных с другими именами
```

Деструктуризация прямо в параметрах функции:

```
function showUser({ name, age, city }) {  
  console.log(`Привет, ${name}! Мне ${age} лет и ты живёшь в ${city}.`);  
}  
  
showUser(person);
```

Функция `showUser` сразу распаковывает нужные свойства объекта `person`, который передаётся ей аргументом.

Объединение объектов

```
const person = {  
  name: "Марат",  
  "last name": "Муслимов",  
  age: 30,  
  city: "Москва",  
};  
  
const job = {  
  position: "Программист",  
  company: "IT-симулятор",  
};  
  
const user = { ...person, ...job };
```

Теперь объект `user` содержит ключи (свойства) обоих объектов — `person` и `job`. Если в объектах встречаются одинаковые ключи, то в объединённом объекте останется значение из того объекта, который был указан последним.

```
const updatedPerson = { ...person, job: "worker", age: 31 };  
console.log(updatedPerson);
```

К новому объекту `updatedPerson` добавлены все ключи объекта `person`, плюс новый ключ `job`, а значение ключа `age` переопределено на 31.

Правила именования функций

- имя начинается с маленькой буквы;

- используется стиль `camelCase`;
 - имя всегда начинается с глагола;
 - если функция что-то возвращает через `return` — используем префикс `get` (например `getUser()`);
 - если функция возвращает `Boolean` — префиксы `is` (это), `has` (имеет), `can` (может): например `isAdmin()`;
 - если функция обрабатывает событие (например клик) — префиксы `handle`, `on`: например `onButtonClick()`.
-

Массивы

Массив — упорядоченная коллекция данных, к элементам которой обращаются по индексу. В отличие от объекта, где используются именованные ключи, в массиве каждое значение имеет свой номер (индекс), начиная с 0. Массив может хранить любые типы данных: числа, строки, логические значения, объекты, функции, переменные.

```
const numbers = [1, 2, 3, 4, 5];
console.log(numbers[3]); // 4 — индексация начинается с 0
```

Способы создания массива:

```
const numbers = [1, 2, 3, 4, 5];
const array = new Array(1, 2, 3);
```

Особенность `new Array(...)`: если написать `new Array(9)`, создастся массив из 9 пустых ячеек, а не один элемент со значением 9.

Длина массива — количество элементов. Узнаётся через свойство `.length`, которое возвращает индекс последнего элемента + 1. Через это же свойство можно и задать длину массива: `numbers.length = 20`;

Как и объект, массив хранится в памяти в виде ссылки.

Объединение массивов

Есть два способа:

1. Через метод `.concat`:

```
const a = [1, 2, 3];
const b = [4, 5, 6];
const c = a.concat(b);
```

2. Через оператор расширения `... (spread)`:

```
const d = [...a, ...b];
```

Через `spread` можно также добавлять отдельные элементы вручную:

```
const e = [...a, 7, ...b, 8];
```

Методы массивов push / pop / unshift / shift

.push() — добавляет элемент в конец массива:

```
const array = [1, 2, 3];
array.push(4); // [1, 2, 3, 4]
```

.pop() — удаляет последний элемент массива:

```
const array = [1, 2, 3];
array.pop(); // [1, 2]
```

.unshift() — добавляет элемент в начало массива:

```
const array = [1, 2, 3];
array.unshift(9); // [9, 1, 2, 3]
```

.shift() — удаляет первый элемент массива:

```
const array = [1, 2, 3];
array.shift(); // [2, 3]
```

Метод map()

Метод map() используется для трансформации массива: он создаёт новый массив того же размера на основе исходного. Поочерёдно берёт каждый элемент исходного массива, передаёт его в функцию-колбэк, получает изменённое значение и записывает его в новый массив.

```
const prices = [100, 250, 500];

// map берёт каждое число и умножает на 2
const doubledPrices = prices.map(price => price * 2);

console.log(doubledPrices); // [200, 500, 1000] — новый массив
console.log(prices); // [100, 250, 500] — старый массив не изменился
```

Полный синтаксис

Функция, которую передают в map(), может принимать три аргумента (обычно используют только первый):

```
const newArray = arr.map((item, index, array) => {
  // item — текущий элемент массива (нужен в 99% случаев)
  // index — порядковый номер текущего элемента (начиная с 0)
  // array — ссылка на исходный массив (нужен редко)
  return item;
});
```

arr и **array** — это один и тот же массив, доступный в разных частях кода под разными именами. Технически array — это ссылка на массив arr.

В чём разница между ними?

- **arr** (внешнее имя) — переменная, которую создали вы сами в основной программе. Доступна везде в вашем коде (глобально или внутри текущей функции).

- `array` (внутреннее имя) — локальная переменная, которую автоматически создаёт сам JavaScript внутри метода `map()`. Существует и доступна только внутри фигурных скобок `map(...)` — как только `map` завершит работу, эта переменная исчезнет.

Аналогия из жизни

Представьте человека по имени Александр (`const arr = "Александр"`). Когда он приходит на работу, начальник обращается к нему по должности: «Сотрудник» — как к аргументу (`item, index, array`) => `{...}`. Внутри офиса (внутри `map`) его зовут Сотрудник, но это всё тот же Александр.

Зачем вообще нужно имя `array`, если есть `arr`?

В простых примерах они взаимозаменяемы — можно вместо `array` писать `arr`, и код будет работать. Но `array` незаменим в двух случаях:

1. Если у массива нет собственного имени (цепочка методов):

```
const result = "яблоко,банан,груша"
  .split(",") // тут создан массив, но у него НЕТ имени!
  .map((item, index, array) => {
    // Как посмотреть на весь массив? Переменной arr не существует!
    // Вот тут и спасает аргумент array.
    return item;
  });
```

2. Для универсальности (чистые функции):

Если вынести колбэк-функцию отдельно, она не будет знать, как назывался исходный массив в вашей программе:

```
// Эта функция универсальна, она не знает о переменной "arr"
function myTransform(item, index, array) {
  return `Элемент ${item} из массива длиной ${array.length}`;
}

const arr = [1, 2, 3];
const users = ['A', 'B'];

arr.map(myTransform); // здесь array будет равен arr
users.map(myTransform); // а здесь array будет равен users
```

Главные отличия от других методов

- `map()` против `forEach()`: `forEach` просто перебирает массив (как цикл `for`), но ничего не возвращает. `map` всегда возвращает новый трансформированный массив.
- `map()` против `filter()`: `filter` отсеивает элементы и может уменьшить длину массива. `map` всегда возвращает массив ровно той же длины, что и оригинал.

Особенность `map()` при работе с массивом объектов: в новом массиве будут храниться ссылки на те же объекты, а не копии их значений. Чтобы не изменить исходный массив, внутри колбэка нужно использовать `spread` — создавая новый объект вместо изменения старого.

Сравнение двух подходов

Действие в коде	Что происходит с объектами?
<code>reader.isRare = true</code>	Переменная reader указывает на старый объект. Вы меняете его оригинал напрямую.
<code>{ ...reader, isRare: true }</code>	Создаётся новый «слепок» объекта. Старый объект остаётся нетронутым.